

SmartLogix

Diego Lazo
Antoane Anticoi

Contenido

FICHA DEL DOCUMENTO

1. INTRODUCCIÓN

- 1.1. Contexto del Problema
- 1.2. Propósito
- 1.3. Ámbito
- 1.4. Arquitectura del Sistema

2. VISIÓN DEL SISTEMA

- 2.1. Descripción General del Sistema
- 2.2. Objetivos del Sistema
- 2.3. Requerimientos Funcionales

3. ESTILOS Y PATRONES ARQUITECTÓNICOS

- 3.1. Justificación del Estilo
- 3.2. Requisitos Funcionales Principales
- 3.3. REQUISITOS No FUNCIONALES
 - 3.3.1 REQUISITOS DE RENDIMIENTO
 - 3.3.2 SEGURIDAD

4. MODELO 4+1 Y VISTAS ARQUITECTÓNICAS

- 4.1. Justificación del Estilo
- 4.2. VISTA DE ESCENARIO

5. DEFINICIÓN DE HISTORIAS DE USUARIO

- 5.1. HISTORIA DE USUARIO: COMPRA DE PRODUCTO
- 5.2. Historia de Administrador: Gestión de Producto

6. PRINCIPIOS DE DISEÑO APLICADO

7. ANÁLISIS DE PATRONES Y ARQUETIPOS (JUSTIFICACIÓN TÉCNICA)

- 7.1. ARQUETIPO: MAVEN MULTI-MODULE
- 7.2. Patrón Arquitectónico: Microservicio con Base de Datos por Servicio
- 7.3. Patrón de Diseño: BFF (Backend For Frontend)
- 7.4. Patrón DTO (Data Transfer Object)
- 7.5. Diagrama UML

8. PLAN DE BRANCHING

9. CARTA GANTT

10. ENLACES A REPOSITORIOS

11. CONCLUSIONES

1. INTRODUCCIÓN

SmartLogix, una solución de software basada en una arquitectura de **microservicios**. El sistema ha sido diseñado para resolver problemas de escalabilidad y mantenibilidad, permitiendo que la gestión de usuarios, el inventario y las órdenes de compra operen de forma independiente pero coordinada.

1.1 Contexto del Problema

Este documento define los requisitos y la arquitectura del sistema SmartLogix, diseñado para optimizar la gestión de inventarios y ventas en entornos distribuidos. Está dirigido al equipo de desarrollo y a la dirección técnica del proyecto.

1.2 Propósito

SmartLogix es un sistema basado en microservicios que permite la gestión automatizada de productos, el control de stock y el registro de órdenes de compra. El sistema garantiza que cada módulo sea independiente, facilitando su mantenimiento y escalabilidad.

1.3 Ámbito

El documento abarca desde la definición de requisitos funcionales hasta la arquitectura lógica detallada, incluyendo la infraestructura local basada en Laragon.

1.4 Arquitectura del Sistema

Se ha implementado una arquitectura de **Microservicios con patrón BFF (Backend For Frontend)**.

- **Servicio Auth:** Gestión de identidad y seguridad JWT.
- **Servicio Inventory:** Gestión de catálogo y stock.
- **Servicio Orders:** Procesamiento de pedidos.
- **BFF:** Orquestador y punto de entrada único.

2. VISIÓN DEL SISTEMA

2.1 Descripción General del Sistema

SmartLogix es un producto de software independiente que utiliza una arquitectura distribuida para evitar puntos únicos de fallo.

2.2 Objetivos del Sistema

1. Centralizar la información de productos y ventas en bases de datos relacionales independientes.
2. Implementar seguridad robusta mediante el estándar JWT.
3. Permitir la consulta rápida de stock a través de una API REST orquestada por un BFF.

2.3 Requerimientos Funcionales

- **ADMIN:** Acceso total al CRUD de inventario, visualización de todas las órdenes y gestión de usuarios.
- **USER:** Capacidad de navegar por el catálogo y generar órdenes de compra personales.

3. ESTILOS Y PATRONES ARQUITECTÓNICOS

3.1 Justificación del Estilo

Se seleccionó el estilo de **Microservicios** debido a la necesidad de escalabilidad horizontal. El patrón **BFF** se justifica por la necesidad de simplificar la comunicación del Frontend (React) con múltiples servicios internos, reduciendo la latencia y facilitando la autenticación centralizada.

3.2 Requisitos Funcionales Principales

- **3.2.1 RF-Auth:** El sistema debe autenticar usuarios y retornar un token JWT válido.
- **3.2.2 RF-Stock:** El sistema debe descontar stock automáticamente al procesar una orden
- **3.2.3 RF-Orders:** El sistema debe permitir la creación de pedidos vinculados a un ID de usuario.

3.3 Requisitos No Funcionales

3.3.1 Requisitos de Rendimiento

- El tiempo de respuesta del BFF para consultas de productos debe ser inferior a 1 segundo en el 95% de las peticiones.
- El sistema debe soportar hasta 50 usuarios concurrentes sin degradación del servicio.

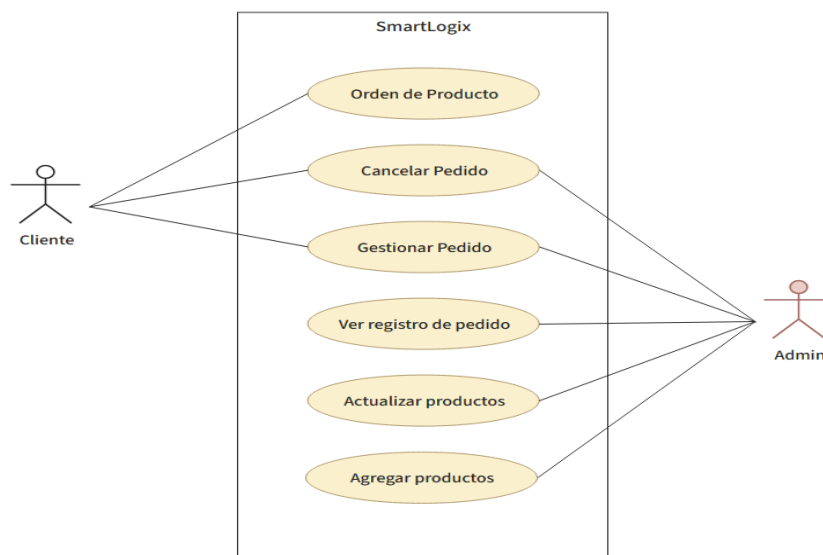
3.3.2 Seguridad

- **JWT:** Empleo de técnicas criptográficas para la firma de tokens de acceso.
- **Logs:** Registro de actividad en consola para el monitoreo de transacciones entre servicios
- **Roles:** Restricción de funcionalidades según el perfil de usuario (RBAC).

4. MODELO 4+1 Y VISTAS ARQUITECTÓNICAS

4.1 Vista de Escenario

- **Propósito:** Describir la interacción del usuario con las funciones clave del negocio.
- **Actores:** Administrador y Cliente.



4.2 Vista Lógica

Describe la descomposición en microservicios: `AuthService`, `InventoryService`, `OrderService` y `BFF`. Cada uno posee su propia base de datos (`authdb`, `inventorydb`, `ordersdb`) para asegurar el desacoplamiento de datos.

5. DEFINICIÓN DE HISTORIAS DE USUARIO

5.1 Historia de Usuario: Compra de Producto

- **Como** Cliente, **quiero** agregar productos al carrito y confirmar mi compra, **para** obtener los artículos que necesito.
- **Criterio de Aceptación:** La orden se crea con estado "PENDING" y el stock se reduce en la base de datos `inventorydb`.

5.2 Historia de Administrador: Gestión de Producto

- **Descripción:** Como administrador, quiero editar un producto para corregir errores de precio.
- **Criterio:** El cambio se refleja inmediatamente en el Frontend y en la DB de inventario.

6. PRINCIPIOS DE DISEÑO APLICADO

Se aplicó el principio de Responsabilidad Única (SRP) de SOLID, donde cada microservicio gestiona un dominio de negocio específico. El uso de Inyección de Dependencias en Spring Boot asegura un código mantenible y testeable.

7. ANÁLISIS DE PATRONES Y ARQUETIPOS (JUSTIFICACIÓN TÉCNICA)

7.1 Arquetipo: Maven Multi-Module

- Se seleccionó este arquetipo para centralizar la gestión de dependencias y asegurar la uniformidad tecnológica en todo el ecosistema. Al utilizar un proyecto "parent", garantizamos que todos los microservicios (Auth, Inventory, Orders) utilicen las mismas versiones de Spring Boot (3.x) y Java (21), eliminando conflictos de librerías y facilitando la compilación integral del sistema con un solo comando.

7.2 Patrón Arquitectónico: Microservicios con Base de Datos

- **Justificación:** Se implementó el patrón de Aislamiento de Datos (Data Sovereignty). Cada microservicio gestiona su propio esquema en MySQL (`authdb`, `inventorydb`, `ordersdb`).
- **Valor Agregado:** Esto evita el "acoplamiento por base de datos", permitiendo que, si el esquema de Inventario cambia, no se rompa el servicio de Órdenes. Además, facilita la escalabilidad: si el tráfico de ventas aumenta, podemos replicar solo el `Orders-Service` en Laragon sin desperdiciar recursos en los demás módulos.

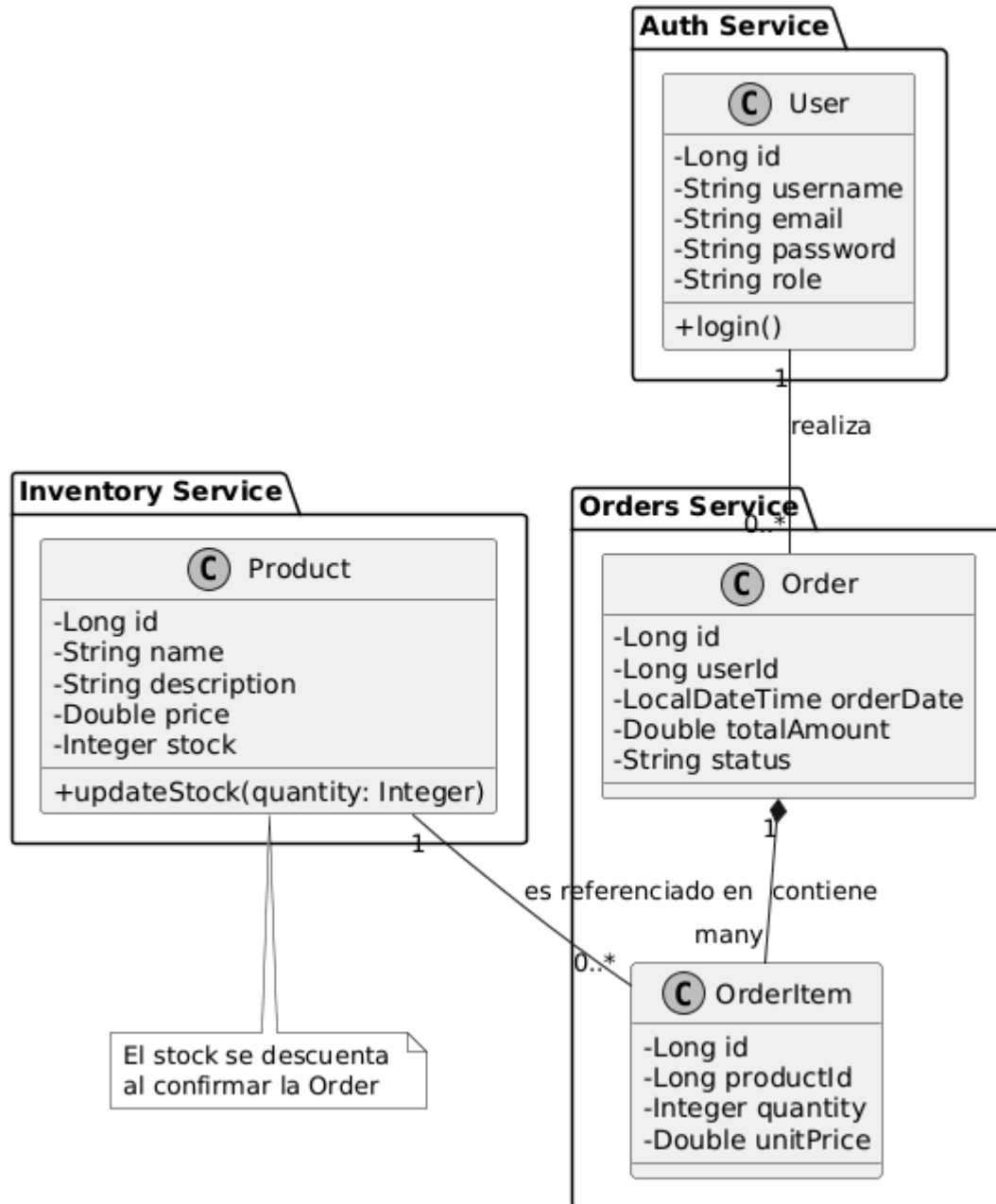
7.3 Patrón de Diseño: BFF (Backend For Frontend)

- **Justificación:** Se utiliza como un **Agregador de Servicios**. En lugar de que el Frontend (React) realice múltiples llamadas asíncronas a distintos puertos, el BFF actúa como punto central.
- **Seguridad:** Centraliza la validación de los tokens JWT. El BFF recibe el token, lo valida una sola vez contra el `Auth-Service` y luego propaga la identidad a los servicios internos, reduciendo la superficie de ataque y simplificando la lógica del cliente.

7.4 Patrón DTO (Data Transfer Object)

- **Justificación:** Implementado para desacoplar el modelo de persistencia (Entidades JPA) del modelo de presentación (JSON). Esto protege la integridad de los datos, evitando que cambios en las tablas de la base de datos afecten directamente al contrato de la API consumida por el Frontend.

7.5 Diagrama de clases UML



8. PLAN DE BRANCHING

- **Rama main (o master):** Es la rama de producción. Solo contiene código estable y probado que ya funciona con Laragon y el Frontend. Nunca se programa directamente aquí.
- **Rama develop:** Es la rama de integración. Aquí se mezclan todas las funciones nuevas de los microservicios antes de pasar a la rama principal.
- **Ramas feature/ (Funcionalidades):** Se crea una rama por cada microservicio o mejora. Ejemplos:
 - **feature/auth-service:** Para el desarrollo del microservicio de seguridad.
 - **feature/inventory-crud:** Para las funciones de stock.
 - **feature/bff-orchestration:** Para el desarrollo del orquestador.
- **Flujo de Trabajo:** Una vez que una **feature** está terminada y probada en el entorno local (VS Code + Laragon), se realiza un *Pull Request* hacia **develop**. Tras una revisión final, se pasa a **main**.

9. CARTA GANTT

Actividad	Duración	Estado
Configuración de Laragon/DB	1 día	Completado
Backend (Microservicios)	2 días	Completado
Frontend (React)	1 día	Completado
Pruebas de Integración	1 día	Completado

10. ENLACES A REPOSITORIOS

1.- **Repositorio principal:** <https://github.com/DiiegooDuoc/SmartLogix>

2.- **Carpetas:**

- **Backend** (<https://github.com/DiiegooDuoc/SmartLogix/tree/main/backend>): En esta carpeta se pueden encontrar los microservicios **Auth**, **Inventory** y **Orders** los cuales son unas de las carpetas principal de este sistema. También se incluye la carpeta BFF (Backend For Frontend) la cual hace el mejor trabajo para disminuir la carga al consultar a un microservicio específico.
- **Frontend** (<https://github.com/DiiegooDuoc/SmartLogix/tree/main/frontend>): En este apartado encontraremos todo lo que seria la parte web de este proyecto, mostrandonos archivos como Dashboard para ir haciendo la interfaz que tendra a pagina web y que se mostrara en tal.

3.- **Ramas del Repositorio:**

- **main** (<https://github.com/DiiegooDuoc/SmartLogix/tree/main>): Esta es la rama principal en la cual se sube el proyecto final, listo para llegar, descargar y probar.
- **develop** (<https://github.com/DiiegooDuoc/SmartLogix/tree/develop>): Esta es la rama para poder hacer pruebas de los cambios realizados y así evitar colapsar el proyecto principal en caso de "X" problema.
- **feature/auth-service** (<https://github.com/DiiegooDuoc/SmartLogix/tree/feature/auth-service>): Esta es una de las ramas enfocadas en cada microservicio, en este caso se creo en base a el servicio de AUTH, la cual esta hecha para subir actualizaciones hechas en los respectivos microservicios. A futuro pueden variar, como por ejemplo agregando una rama "feature/inventory-service".

11. CONCLUSIONES

La implementación de SmartLogix bajo una arquitectura de microservicios cumple satisfactoriamente con los requerimientos de escalabilidad planteados. La integración con Laragon permitió un entorno de desarrollo eficiente, mientras que el uso de Spring Boot garantiza una robustez técnica alineada con los estándares actuales de la industria.